

Informe Técnico Analítico - “Sistemas de Semáforos Inteligentes”

Integrantes del grupo: Aponte Eduardo Jose

Caceres Lourdes Gabriela

Gómez Susana Nelida

Gonzalez Vanessa Marlene

Los Santos Misael Adonias

Tema investigado: HASKELL

En este documento, dejamos registrado los fundamentos del diseño funcional adoptado por el grupo para la realización del “Sistemas de Semáforos Inteligentes”.

- Nuestros motivos para integrar la librería **cl-json** a nuestro código.
- La bitácora de depuración de cada uno de nosotros
- Las conclusiones que llegamos al reimplementar la funciones **transición** y **timer** al lenguaje Haskell

Introducción al proyecto

El objetivo de este proyecto consiste en el diseño e implementación del núcleo lógico inmutable para un Sistema de Semáforos Inteligentes en intersecciones urbanas críticas. El propósito primordial de este software es proveer una herramienta automatizada de simulación y análisis métrico temporal que optimice el flujo vehicular y la seguridad vial, proporcionando al operador informes estadísticos detallados ante diversos escenarios operacionales.

El sistema está construido bajo el paradigma funcional utilizando el lenguaje ANSI Common Lisp. Para la abstracción y gestión de las constantes, se integra la librería de serialización **cl-json** provista por el ecosistema Quicklisp.

Especificaciones técnicas

El usuario puede interactuar con el sistema a través de una Interfaz de Línea de Comandos (CLI) estructurada mediante un menú jerárquico de opciones iterativo. Las principales herramientas del software se dividen en los siguientes requerimientos específicos:

1. **Modelado y Validación de Transiciones:** Simulación del cambio de luces del semáforo, discriminando secuencias válidas de aquellas inválidas.
2. **Sincronización Temporal (Timer):** Determinación exacta del color activo en un instante de tiempo Unix específico (**timestamp**).
3. **Auditoría Histórica de Eventos:** Registro secuencial de las transiciones operadas en el sistema para la generación de informes cronológicos de ejecución.
4. **Análisis de Optimización de Ciclos:** Cálculo de la duración total (en segundos) de un ciclo completo de luces y su evaluación en relación con los rangos óptimos de la psicología del conductor.

5. **Planificación Temporal de Frecuencias:** Computación del número entero de ciclos íntegros que se ejecutarán dentro de un intervalo parametrizado en minutos.
6. **Análisis Estadístico de Distribución Lumínica:** Generación de un informe de asignación porcentual de tiempo por cada color en un bloque estático de una hora de simulación.

El semáforo inteligente opera bajo una estructura tricolor estándar (Rojo, Amarillo y Verde) cuyos tiempos de permanencia base están asignados en el archivo de configuración `config.json` con los siguientes valores:

- Estado en Rojo (rojo): 90 segundos.
- Estado en Amarillo (amarillo): 6 segundos.
- Estado en Verde (verde): 120 segundos.
- Estado Intermitente (intermitente): 3 segundos de transición de seguridad (incorporado en la Iteración 2).

Se define formalmente como Ciclo Semafórico Completo a la secuencia de transiciones Rojo -Verde - Amarillo - Intermitente. La sincronización cronológica global del sistema se rige mediante el estándar de tiempo Unix (Epoch Time), procesando la entrada como un tipo de dato entero plano para calcular de forma no destructiva el estado activo..

Entorno de Ejecución y Dependencias

Para la correcta ejecución y despliegue del sistema de semáforos, el entorno debe satisfacer los siguientes requisitos:

1. Consola del Sistema (CLI): La interacción se realiza mediante una terminal estándar de comandos (ej. `cmd` en Windows, `bash/zsh` en entornos Unix/Linux).
2. Entorno de Ejecución Common Lisp: Es mandatorio contar con un compilador/intérprete que provea un bucle de lectura, evaluación (REPL). Se recomienda explícitamente el uso de SBCL o CLIPS.
3. Gestión de Dependencias Externas: El sistema requiere la presencia previa del gestor de paquetes Quicklisp configurado en el directorio del usuario. El código fuente de la aplicación automatiza de forma transparente la inicialización del entorno (`setup.lisp`) y la descarga adaptativa de la librería de serialización `:cl-json`.

Fase 1: Diseño e implementación del Sistema de Semáforos Inteligentes (en common LISP)

REQUERIMIENTO 1: Estados de Transición

Objetivo: Modelar y validar de forma determinista el cambio de un estado luminoso a otro, impidiendo secuencias de transición que violen las reglas de seguridad vial.

Funciones que lo integran:

- `transicion (color-actual cambiar-a)`

Conexión entre funciones: Es una función atómica e independiente dentro del módulo core. No invoca funciones auxiliares ni de otros requerimientos.

Naturaleza: Pura. A idénticos argumentos de entrada produce exactamente el mismo valor de retorno, careciendo por completo de efectos secundarios (*side-effects*).

Parámetros de Entrada:

- **color-actual**: Tipo **Símbolo** (SYMBOL). Representa el estado actual del semáforo (Valores válidos: 'en-rojo, 'en-amarillo, 'en-verde).
- **cambiar-a**: Tipo **Símbolo** (SYMBOL). Representa el color destino hacia el cual se intenta transicionar (Valores válidos: 'rojo, 'amarillo, 'verde).

Retorno: Tipo **Lista** (LIST).

- Caso Exitoso: Retorna una lista con dos elementos: el símbolo del estado origen y una cadena descriptiva de la acción, por ejemplo: ('en-rojo "cambiar-a-verde").
- Caso Fallido / Control de Errores: Retorna una lista de control de fallas mediante cláusulas del operador **cond**, por ejemplo: ('error "color-actual-invalido").

REQUERIMIENTO 2: Temporizador Automático (Timer)

Objetivo: Sincronizar el estado activo del semáforo con el tiempo actual (en *Epoch time*) mediante cálculo aritmético modular, permitiendo deducir la luz encendida en cualquier instante temporal sin necesidad de mutar variables de estado.

Funciones que lo integran:

- **calcular-timer** (tiempo)
- **calcular-rem** (tiempo rojo verde amarillo intermitente)
- **compararRem** (resto rojo verde amarillo intermitente)

Conexión entre funciones:

1. **calcular-timer**: Funciona como la interfaz pública del módulo. Valida el tipo de dato de entrada, ejecuta cuatro llamadas a la función externa **obtener-color** para recuperar los tiempos del archivo **.json** e invoca a **calcular-rem**.
2. **calcular-rem**: compara el resto (rem) del tiempo Unix con respecto a la duración total del ciclo acumulado (incluyendo los tres periodos de intermitencia de la Iteración 2) y transfiere este valor a **compararRem**.
3. **compararRem**: Evalúa mediante estructuras condicionales el rango temporal al que pertenece el residuo calculado.

Naturaleza: **Pura**, ya que no altera estados globales ni produce escrituras destructivas.

Parámetros de Entrada:

- **tiempo / resto**: Tipo **Entero** (INTEGER). Representa el timestamp Unix o el residuo rem de la división.
- **rojo, verde, amarillo, intermitente**: Tipo **Entero** (INTEGER). Valores en segundos recuperados de la configuración.

Retorno: Tipo **Símbolo** (SYMBOL) o **Cadena** (STRING).

- Retorna un símbolo que representa el estado exacto del semáforo ('en-rojo, 'rojo-intermitente, 'en-verde, 'verde-intermitente, 'en-amarillo, 'amarillo-intermitente) o un string en caso de error estructural.

REQUERIMIENTO 3: Sistema de Auditoría

Objetivo: Capturar, estructurar, formatear y persistir el historial cronológico de cambios de estado del sistema semafórico, proveyendo salidas tanto en la terminal interactiva como en un almacenamiento físico persistente.

Funciones que lo integran:

- registrar-evento (timestamp estado-anterior estado-nuevo historial)
- mostrar-evento (evento)
- mostrar-historial (historial)
- evento-a-texto (evento)
- historial-a-texto (historial)
- guardar-informe (historial)
- informe (historial)

Conexión entre funciones: El módulo se divide dos sub-estructuras acopladas por flujos de datos:

- Flujo de Consola: mostrar-historial procesa recursivamente la lista del historial extrayendo cada nodo con car para enviarlo a mostrar-evento.
- Flujo de Persistencia en Disco: guardar-informe abre un canal físico de escritura (with-open-file), e inyecta el string generado por la función constructora informe. Esta última invoca a historial-a-texto la cual, mediante recursión constructiva (concatenate 'string), une los strings individuales generados por evento-a-texto procesando la lista invertida (reverse).

Naturaleza: Mixta.

- registrar-evento, evento-a-texto, historial-a-texto e informe son **Puras**.
- mostrar-evento, mostrar-historial y guardar-informe son **Impuras**, dado que producen efectos secundarios en el entorno del sistema (I/O por pantalla con format-t y mutación de archivos físicos en disco).

Parámetros de Entrada:

- timestamp: **Entero** (INTEGER).
- estado-anterior, estado-nuevo: **Símbolos** (SYMBOL).
- evento: **Lista** conteniendo (timestamp estado-ant estado-nue).
- historial: **Lista de Listas** (LIST de sub-listas) que representa la estructura de datos acumulativa.

Retorno: * registrar-evento devuelve una nueva **Lista de Listas** con el nodo insertado en el tope mediante cons.

- Las funciones de impresión devuelven **NIL** o cadenas vacías al agotar la recursión.
- Las funciones transformadoras devuelven tipos **Cadena** (STRING).

REQUERIMIENTO 4: Análisis de Ciclos Semafóricos

Objetivo: Calcular el tiempo total en segundos de un ciclo completo de tradiciones del semáforo y emitir una “recomendación” de optimización vial basado en la ingeniería de tráfico y la psicología del conductor.

Funciones que lo integran:

- duracion ()
- recomendacion-ciclo (duracion)

Conexión entre funciones: La salida de la función duracion está diseñada para alimentar directamente la entrada de recomendacion-ciclo. Ambas dependen internamente de la consulta de constantes mediante obtener-color.

Naturaleza: Pura. No mutan variables y operan de forma matemática determinista.

Parámetros de Entrada:

- **duracion:** no recibe datos de entrada como parámetros. Para **recomendación-ciclo**, recibe un tipo **Numérico / Entero (INTEGER)** que representa el total de segundos de la revolución.

Retorno:

- **Duración:** Un **número entero (INTEGER)** resultado de la suma de todas las luces e intermitencias registradas en el JSON.
- **recomendación-ciclo:** Un mensaje en pantalla (**STRING**) de la evaluación de la condiciones del rango establecido ("Ciclo demasiado largo", "Ciclo demasiado corto", "Ciclo óptimo"), o interrumpe la ejecución mediante señales de desborde con **error** si las precondiciones tipadas fallan.
-

REQUERIMIENTO 5: Planificación Temporal

Objetivo: Determinar cuantitativamente cuántos ciclos completos tendrán lugar durante un bloque temporal parametrizado por el usuario en minutos.

Funciones que lo integran:

- cantidad-ciclos (minutos)

Conexión entre funciones: Es una función independiente, pero acoplada con el archivo de configuración externo; invoca repetidamente a **obtener-color** para obtener la duración.

Naturaleza: Pura.

Parámetros de Entrada:

- minutos: Tipo **Entero (INTEGER)** representa el bloque temporal (en minutos).

Retorno: Un **Elemento Único Entero (INTEGER)**. Esto se garantiza mediante la aplicación de la función primitiva de truncamiento hacia abajo **floor**, la cual descarta las fracciones de ciclo.

REQUERIMIENTO 6: Informe de Distribución Temporal

Objetivo: Determinar el porcentaje de tiempo que cada luz (de forma individual) permanecerá activa durante un intervalo estándar de una hora (3600 segundos), considerando escenarios de ciclos incompletos (residuos).

Funciones que lo integran:

- porcentaje-color ()
- calcular-rest (resto rojo verde amarillo actual)
- calcular (color resto ciclos)

Conexión entre funciones:

1. porcentaje-color: Funciona como el componente central del flujo. Calcula la duración total de un ciclo, determina cuántos ciclos completos caben en una hora (3600 segundos) y obtiene el residuo de tiempo restante.
2. Flujo de resolución: Mediante composición funcional, la función invoca a **calcular-rest** para asignar los segundos del ciclo incompleto al color correspondiente (evaluado según las prioridades de la estructura condicional **cond**).
3. Salida: Finalmente, transfiere tanto el residuo asignado como el total de ciclos completos a la función aritmética **calcular** para el procesamiento final.

Naturaleza: Pura.

Parámetros de Entrada:

- **porcentaje-color**: Ninguno (asume la constante de 1 hora del dominio del problema).
- **resto, rojo, verde, amarillo, color, ciclos**: Tipos **Entero** (INTEGER).
- **actual**: Tipo **Símbolo** (SYMBOL). Clave de mapeo condicional.

Retorno: Tipo **Lista de Celdas Cons** (LIST de CONS pairs). Devuelve una lista que empareja el símbolo del color con su cálculo porcentual exacto en punto flotante.

Fase 2: Integración del gestor de paquetes Quicklisp - Librería cl-json

En el marco de la Fase 2 del proyecto, y en cumplimiento con el requisito de integrar el gestor de paquetes Quicklisp, se seleccionó la librería externa `cl-json`. Esta decisión de diseño responde a la necesidad de desarticular las constantes temporales del semáforo (parámetros de negocio) de la lógica ejecutable del código. Mediante la externalización de estas variables en el archivo `config.json`, el sistema adquiere flexibilidad y dinamismo, reduciendo el riesgo de dependencia de datos estáticos dispersos en múltiples funciones del sistema. Asimismo, debido a la naturaleza modular y puramente funcional del código base desarrollado en la Fase 1, la integración de este componente presentó un bajo impacto de refactorización, preservando la estructura y comportamiento de las funciones ya validadas.

La manipulación y lectura de esta estructura en el entorno de Lisp se resuelve mediante la implementación de una capa de abstracción de datos compuesta por dos funciones auxiliares complementarias:

1. **leer-config**: Se encarga de la apertura del flujo de entrada físico (`config.json`) mediante el macro orientado a recursos `with-open-file`. Delega en la librería `cl-json:decode-json` la deserialización sintáctica del archivo, transformando los pares clave-valor en una estructura nativa de lista de asociación (*A-list*) de Common Lisp.
2. **obtener-color**: Actúa como la interfaz pública de la lista obtenida del archivo `config`. Recibe como parámetro un símbolo que representa la clave del color (`:ROJO`, `:AMARILLO`, `:VERDE`, `:INTERMITENTE`) y efectúa una búsqueda lineal indexada utilizando el predicado estructural `ASSOC` sobre la *A-list* devuelta por `leer-config`. Para evitar el retorno del par asociativo completo (`CLAVE . VALOR`), se aplica la función selectora `cdr`, abstrayendo al resto del sistema y devolviendo únicamente el elemento entero que representa el tiempo en segundos.

La combinación de ambas funciones transforma el acceso a los datos, permitiendo que el resto de los módulos (como `calcular-timer`, `duracion`, e `informe-porcentaje`) utilicen los valores de forma dinámica y centralizada, ocultando los detalles de la implementación física del archivo y garantizando la modularidad del software.

Fase 3: Reimplementación y comparación con Haskell

Haskell es un lenguaje de programación multipropósito y puramente funcional, reconocido por su tipado estático fuerte y su inmutabilidad de datos. Es una herramienta fundamental para entender nuevos paradigmas de desarrollo, evitando estados mutables y enfocándose en expresar la lógica a través de funciones matemáticas.

Características

- **Funcional Puro**: Las funciones matemáticas no tienen efectos secundarios. La misma entrada siempre producirá exactamente la misma salida.
- **Tipado Estático con Inferencia**: El compilador detecta errores antes de ejecutar el programa. Además, es muy inteligente deduciendo los tipos de datos, por lo que rara vez necesitas declararlos explícitamente.

- Evaluación Perezosa (Lazy Evaluation): Haskell no calcula los valores hasta que realmente los necesita, lo que permite crear estructuras de datos infinitas y mejorar el rendimiento.
- Inmutabilidad: Una vez que se asigna un valor a una variable, esta no puede cambiar, lo que hace al código más seguro y fácil de depurar.

Ventajas

- Código muy conciso y fácil de mantener,
- Excelente manejo de la concurrencia y paralelismo.
- Altamente valorado por la seguridad y prevención de errores en tiempo de compilación.
- El código del software de Haskell es breve, claro y fácil de mantener.
- Menos propensas a errores y ofrecen una gran fiabilidad.
- La brecha semántica entre el programador y el lenguaje es mínima.

Desventajas

- Curva de aprendizaje empinada para quienes vienen de lenguajes como Python o C++.
- Ecosistema menor en comparación con lenguajes de propósito general masivo
- Interfaz con hardware o entrada/salida (I/O) más compleja debido a su pureza matemática.
- Es utilizado por gigantes tecnológicos y corporaciones en criptomonedas, inteligencia artificial, finanzas y desarrollo web.

Diferencia de Haskell con otros lenguajes de programación

Es un lenguaje de programación puramente funcional, Haskell es muy distinto de muchos otros lenguajes. Se diferencia en particular de los lenguajes que se orientan al paradigma imperativo. Los programas escritos en lenguaje imperativo ejecutan secuencias de instrucciones. Incluso durante la ejecución, el estado de estas declaraciones puede cambiar, por ejemplo, al modificar las variables. Las estructuras de flujo de control garantizan que las instrucciones se puedan ejecutar repetidamente.

Reimplementación de las funciones **transición** y **timer** al lenguaje Haskell

La experiencia de aprendizaje con Haskell resultó inicialmente compleja debido a sus diferencias sintácticas respecto de la mayoría de los lenguajes de programación modernos y de uso masivo. Su enfoque puramente funcional introduce conceptos y formas de resolución de problemas que requieren un cambio de paradigma para quienes poseen experiencia previa en lenguajes imperativos u orientados a objetos.

No obstante, se observaron similitudes conceptuales con Common Lisp, especialmente en el uso de funciones puras, la ausencia de efectos secundarios y la importancia de la composición funcional como mecanismo principal para la construcción de soluciones. Estos aspectos facilitaron la comprensión de algunos conceptos fundamentales, ya que ambos lenguajes promueven un enfoque declarativo y matemático para el desarrollo de programas.

A pesar de estas similitudes, la adaptación a la sintaxis de Haskell y a las herramientas necesarias para su ejecución y desarrollo demandó un tiempo considerable. La configuración del entorno, el sistema de tipos y las particularidades de la escritura de funciones representaron una curva de aprendizaje más pronunciada de lo esperado. En consecuencia, el proceso de implementación resultó igual de demandante y desafiante que en Lisp, aunque permitió adquirir una comprensión más profunda de los principios de la programación funcional.

Common Lisp tiene un tipado dinámico. Haskell tiene un tipado estático estricto. ¿Cómo modelaron los colores del semáforo? Expliquen qué es un Algebraic Data Type (ADT) y cómo el compilador de Haskell impide pasar un estado inválido antes de que el programa se ejecute.

¿Qué es un Algebraic Data Type (ADT)?

Un ADT (**Tipo de Datos Algebraico**) es un tipo de dato personalizado que se crea combinando otros tipos. Se llaman "algebraicos" porque se forman mediante operaciones similares a la suma y la multiplicación algebraicas:

1. **Tipos Suma (unión):** El tipo puede ser una cosa **O** otra.
2. **Tipos Producto (conjunción):** El tipo contiene una cosa **Y** otra.

¿Cómo se modelaron los colores del semáforo en este código?

Los colores y estados de transición se modelaron de forma **implícita** como cadenas de caracteres (String, que en Haskell es un sinónimo de lista de caracteres [Char]) mapeados directamente dentro de las expresiones lógicas de la función `compararRem`. Los tiempos de duración (90, 6, 120, 3) se encuentran acoplados de manera fija (*hardcoded*) en la función de nivel superior `calcuTimer`.

¿Cómo impide el compilador (GHC) pasar un estado inválido?

A diferencia de Common Lisp, donde los errores de tipo o símbolos inválidos (como `color-actual-invalido` o `transicion-no-valida`) se detectan **en tiempo de ejecución** (cuando el programa ya está corriendo y falla en los bloques `cond`), **Haskell opera en tiempo de compilación**.

El compilador de Haskell (GHC) utiliza un mecanismo llamado **Verificación Estática de Tipos y Análisis de Exhaustividad** (*Exhaustiveness Checking*):

- **Seguridad en la Entrad:** Gracias al tipado estático estricto, GHC garantiza la total inmunidad del temporizador contra datos de tiempo corruptos en la entrada. Si otra sección del sistema intenta invocar a `calcuTimer` pasándole un argumento inválido (como un string "14:30" o un flotante 178145.2), el compilador detendrá la construcción del binario inmediatamente con un error de tipo (*Static Type Mismatch*). El programa inválido jamás llegará a ejecutarse.
- **Falla de Seguridad en la Salida:** Debido a que el tipo de retorno es un genérico String, **GHC es completamente incapaz de detectar un estado operativo inválido**. Si hay un error de tipeo (por ejemplo, escribir "en-roho" en vez de "en-rojo"), o si una función de control recibe el resultado de `calcuTimer`, para el compilador cualquier cadena de texto es matemáticamente válida. No hay control semántico en tiempo de compilación sobre el dominio del semáforo.

Cómo afecta la Evaluación Perezosa (Lazy Evaluation) de Haskell al procesamiento de los tiempos del temporizador si tuvieran un flujo infinito de eventos de tráfico.

La evaluación perezosa es una técnica de programación en la que una expresión o una función no se evalúa hasta que no sea necesaria. La expresión se evalúa sólo cuando los resultados se necesitan para una operación en particular, en lugar de evaluarla de inmediato. Esto se usa para mejorar el rendimiento y la eficiencia del programa, al evitar evaluar expresiones innecesarias.

En la mayoría de los lenguajes tradicionales (incluyendo Common Lisp), se utiliza la Evaluación Estricta. Esto significa que cuando pasas una expresión como argumento a una función, el compilador o intérprete la calcula por completo antes de ejecutar el cuerpo de la función.

En cambio, Haskell utiliza Evaluación Perezosa por defecto. Haskell no evalúa una expresión hasta que su resultado sea estrictamente necesario para mostrar una salida o tomar una decisión. En lugar de calcular el valor inmediatamente, el compilador crea una estructura interna llamada "Thunk" (un marcador de posición o promesa de evaluación). El Thunk contiene la fórmula matemática y las referencias necesarias para calcular el valor sólo si y cuando otra parte del programa lo demande.

Si haskell tuviera que procesar un flujo infinito de eventos de tráfico, cada elemento del flujo se almacenará en memoria como un thunk (un puntero a la computación que lo genera) y solamente se van a evaluar las expresiones hasta que sea estrictamente necesario para producir un resultado (como mostrarlo en pantalla).

Bibliografía

- [Haskell MOOC](#)
- [Haskell: el lenguaje de programación funcional](#)
- [¿Por qué Haskell?](#)
- [Curso Haskell 2026](#)
- [aprende haskell - youtube](#)

Bitácoras de Depuración

Bitácora de Depuración - Requerimiento 3 - Auditoría de Transiciones Semafóricas e Intermitencias

OBJETIVO: Desarrollar una función capaz de validar y ejecutar transiciones entre estados del semáforo, garantizando que únicamente se permitan cambios válidos entre colores y modos de operación. Posteriormente se amplió la funcionalidad para incorporar los estados de intermitencia requeridos por las extensiones del proyecto.

Error 1: VALIDATION-ERROR - Validación incompleta de colores permitidos

Descripción del error: La función transicion verificaba que el color actual fuera uno de los tres estados básicos:

```
'en-rojo  
'en-amarillo  
'en-verde
```

Sin embargo, al incorporar los modos de intermitencia exigidos por las extensiones, las nuevas entradas:

```
'en-rojo-intermitente  
'en-amarillo-intermitente
```

eran rechazadas como inválidas., tuve que actualizar la validación par que contemple las intermitencias

```
Terminal - zro3@zro3-Smart-E24: ~/Desktop

File Edit View Terminal Tabs Help

*** - SYSTEM::READ-EVAL-PRINT: variable PALGOT.LISP has no value
The following restarts are available:
USE-VALUE :R1 Input a value to be used instead of PALGOT.LISP.
STORE-VALUE :R2 Input a new value for PALGOT.LISP.
ABORT :R3 Abort main loop
Break 1 [2]> palgit.lisp

*** - SYSTEM::READ-EVAL-PRINT: variable PALGIT.LISP has no value
The following restarts are available:
USE-VALUE :R1 Input a value to be used instead of PALGIT.LISP.
STORE-VALUE :R2 Input a new value for PALGIT.LISP.
ABORT :R3 Abort debug loop
ABORT :R4 Abort main loop
Break 2 [3]> (load "palgit.lisp")
;; Loading file palgit.lisp ...
;; Loaded file palgit.lisp
#P"/home/zro3/Desktop/palgit.lisp"
Break 2 [3]> ^[[200~;AUDITORIA - REQUERIMIENTO 3

*** - READ from
#<INPUT CONCATENATED-STREAM #<INPUT STRING-INPUT-STREAM>
#<IO TERMINAL-STREAM>>
: illegal character #\Escape
The following restarts are available:
ABORT :R1 Abort debug loop
ABORT :R2 Abort debug loop
ABORT :R3 Abort main loop
Break 3 [4]> (setq historial
(registrar-evento
1778895600
'en-rojo
'en-verde
nil))
((1778895600 EN-ROJO EN-VERDE))
Break 3 [4]> ^[[200~(setq historial

*** - READ from
#<INPUT CONCATENATED-STREAM #<INPUT STRING-INPUT-STREAM>
#<IO TERMINAL-STREAM>>
: illegal character #\Escape
The following restarts are available:
ABORT :R1 Abort debug loop
ABORT :R2 Abort debug loop
ABORT :R3 Abort debug loop
ABORT :R4 Abort main loop
Break 4 [5]> (setq historial
(registrar-evento
1778895720
'en-verde
'en-amarillo
historial))
((1778895720 EN-VERDE EN-AMARILLO) (1778895600 EN-ROJO EN-VERDE))
Break 4 [5]> (informe historial)
NIL
Break 4 [5]> (probe-file "informe-ejecucion-semaforo.txt")
#P"/home/zro3/Desktop/informe-ejecucion-semaforo.txt"
Break 4 [5]> 
```

Imagen de prueba en terminal linux, tratando de ingresar datos manuales

Error 2: CASOS DE PRUEBA - FORMATO DE DATOS USADOS VS DATOS GENERADOS

Descripción del error: La carga de datos para pruebas use una función aux que me “generaba eventos” que luego eran leídos por la función “registrar-evento”

(defun simular-pruebas-auditoria ()

(let* ((historial-0 nil)

(historial-1

(registrar-evento

1778895600

'en-rojo

'en-verde

historial-0))

(historial-2

(registrar-evento

1778895720

'en-verde

'amarillo-intermitente

historial-1))

(historial-3

(registrar-evento

1778895723

'amarillo-intermitente

'en-amarillo

historial-2))

en un principio era algo asi:

```
(with-open-file
  (stream
    "informe-ejecucion-semaforo.txt"
    :direction :output
    ...)
```

luego escribir-eventos recibia el archivo abierto lo cambie por:

```
(defun historial-a-texto (historial)
  (cond
    ((null historial) "")
    (t
     (concatenate
      'string
      (evento-a-texto (car historial))
      (historial-a-texto (cdr historial))))))
```

que generaba una lista , asi el programa operaba con las listas y al final lo pasaba al archivo con:

```
(defun informe-manual (historial)
  (let ((stream
        (open "informe-ejecucion-semaforo.txt"
              :direction :output
              :if-exists :supersede
              :if-does-not-exist :create)))
    (format stream
      "Informe de Ejecución del Sistema Semafórico~%")
    (format stream
      "=====~%~%")
    (escribir-eventos
     (reverse historial)
     stream)
    (format stream
      "~%--- Fin del Informe ---~%")
    (close stream)))
```

en este caso al principio use supersede , pero luego me di cuenta que si se repetia varias veces el programa ,los nuevos registros remplazaban los anteriores , y no tenia sentido poara mi , asi que cambie el supersede por el append para que mantenga los registros si encontraba el archivo con el nombre el archivo generado correctamente

Solución: modificar (completar) la condicion if

Bug 2: [Error de sintaxis en el ingreso de números decimales]

```
T
* (duracion-ciclo 37,3)

debugger invoked on a SB-INT:SIMPLE-READER-ERROR in thread
#<THREAD tid=10892 "main thread" RUNNING {1104020103}>:
  Comma not inside a backquote.

  Stream: #<SYNONYM-STREAM :SYMBOL SB-SYS:*STDIN* {11000418A3}>

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [ABORT] Exit debugger, returning to top level.

(SB-IMPL::COMMA-CHARMACRO #<SYNONYM-STREAM :SYMBOL SB-SYS:*STDIN* {11000418A3}> #<unused argument>)
0] █
```

Error: SB-INT:SIMPLE-READER-ERROR: Comma not inside a backquote.

Motivo: El error ocurre porque, en Lisp, los números decimales requieren el uso de punto (.) como separador (ej: 37.3) y no la coma, que es estándar en la notación matemática de muchos países hispanohablantes.

Además, el intérprete (Reader) utiliza la coma (,) como un carácter especial reservado para operaciones de *backquote* (evaluación selectiva dentro de listas). Al intentar ejecutar (duracion-ciclo 37,3), el lector encontró la coma en un contexto donde esperaba un argumento numérico o un cierre de paréntesis

Solución: Se realizó la corrección semántica del input (entrada), reemplazando la coma por el punto decimal (37.3). De esta forma se validó que el lector de Common Lisp procesa correctamente 37.3 como un número de punto flotante, evitando el error.

```
41 ) ;fin

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

PS E:\tp_Integrador2026_grupo17\codigoLISP_TpINTEGRADOR> sbcl
This is SBCL 2.6.4, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>.

SBCL is free software, provided as is, with absolutely no warranty.
It is mostly in the public domain; some portions are provided under
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (load "requerimiento4.1.lsp")
T
* (recomendacion-ciclo 37.3)
"Optimo"
* █
```

Bug 3: error en la evaluación del tipo de datos ingresados (negativos - no numéricos)

```

22      );fin cond
23    );fin
24
25    ::agregar validacion de datos (que sea un numero - el numero puede ser entero o float)

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN **TERMINAL** PUERTOS

```

PS E:\tp_Integrador2026_grupo17\codigoLISP_TpINTEGRADOR> sbcl
This is SBCL 2.6.4, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>.

SBCL is free software, provided as is, with absolutely no warranty.
It is mostly in the public domain; some portions are provided under
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (load "requerimiento4.lsp")
T
* (recomendacion-ciclo -37)
"Demasiado corto"
* 

```

Error: No existe un error como tal, e incluso la función evalúa de forma correcta, sin embargo este es un caso que no se puede dar: por ejemplo (duracion-ciclo -37) se pasa un valor negativo, lo cual no tiene sentido que el tiempo de un ciclo sea un valor negativo.

Motivo: Falta de restricciones y validación de datos de entrada para distintos casos.

Solución: Incluir en ambas funciones la validación correspondiente para mantener la integridad de los datos ingresados como parámetros:

Validación 1: que el argumento sea un número, ya sea entero o decimal.

Validación 2: que el argumento sea un valor positivo.

```

(defun duracion-ciclo (ciclo-seg)
  (cond ((not (numberp ciclo-seg)) (error "Tipo de dato incorrecto. Debe ser numérico"))
        ((not (plusp ciclo-seg)) (error "El numero debe ser mayor o igual a 0 (positivo)"))
        (t (if (and (>= ciclo-seg 35)(<= ciclo-seg 150)) t nil ))
  ); fin cond
);fin

```

Salidas esperadas:

- Al pasar un argumento no numérico:

```

* (duracion-ciclo 'r)

debugger invoked on a SIMPLE-ERROR in thread
#<THREAD tid=13424 "main thread" RUNNING {1104020103}>:
  Tipo de dato incorrecto. Debe ser numérico

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [ABORT] Exit debugger, returning to top level.

(DURACION-CICLO R)
source: (ERROR "Tipo de dato incorrecto. Debe ser numérico")
0] 

```

- Al pasar un valor negativo:

```

* (recomendacion-ciclo -35)

debugger invoked on a SIMPLE-ERROR in thread
#<THREAD tid=13424 "main thread" RUNNING {1104020103}>:
  El numero debe ser mayor o igual a 0 (positivo)

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [ABORT] Exit debugger, returning to top level.

(DURACION-CICLO -35)
source: (ERROR "El numero debe ser mayor o igual a 0 (positivo)")
0] 

```

Bug 4: Error en el número de argumento al llamar la función

```

* (recomendacion-ciclo 34 35)

debugger invoked on a SB-INT:SIMPLE-PROGRAM-ERROR @1000C597C9 in thread
#<THREAD tid=11112 "main thread" RUNNING {1104020103}>:
  invalid number of arguments: 2

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [REPLACE-FUNCTION] Call a different function with the same arguments
  1: [CALL-FORM          ] Call a different form
  2: [ABORT              ] Exit debugger, returning to top level.

(RECOMENDACION-CICLO 34 35) [external]
  source: (DEFUN RECOMENDACION-CICLO (SEG-CICLO)
    (COND
      ((NOT (NUMBERP SEG-CICLO))
        (ERROR "Tipo de dato incorrecto. Debe ser numérico"))
      ((NOT (PLUSP SEG-CICLO))
        (ERROR "El numero debe ser mayor o igual a 0 (positivo)"))
      (T
        (COND ((DURACION-CICLO SEG-CICLO) "Optimo")
              ((< SEG-CICLO 35) "Demasiado corto")
              (T "Demasiado largo")))))

```

Error: SB-INT:SIMPLE-PROGRAM-ERROR: invalid number of arguments: 2

Motivo: El error se produjo al intentar invocar la función (recomendacion-ciclo 34 35), pasando dos argumentos numéricos cuando la definición de la función (ambas funciones) solo acepta un parámetro (seg-ciclo). El compilador verifica estrictamente que la aridad de la llamada coincida con la definición; al recibir un argumento demás, el sistema invoca al *debugger* como medida de seguridad para evitar una ejecución indeterminada.

Solución: Se corrigió la llamada en el REPL, pasando un único valor numérico válido (por ejemplo, (recomendacion-ciclo 35)).

VERSIÓN 2:

La función duracion-ciclo retorna el cálculo de la duración total, en segundos, de un ciclo completo (la suma de los segundos de cada color, de forma estática) la cual es utilizada en la función recomendacion-ciclo para evaluar si se encuentra dentro de los parámetros de la psicología del conductor mostrando un mensaje ilustrativo.

```

4 ;; =====
5 ;; FUNCIÓN: duracion-ciclo
6 ;; NATURALEZA: Pura (no produce efectos secundarios)
7 ;; ESTRATEGIA: Operacion aritmética (suma)
8 ;; IMPACTO: No destructiva
9 ;; =====
10 (defun duracion-ciclo ()
11   (+ 90 6 120))
12 );fin
13 ;; =====
14 ;; FUNCIÓN: recomendacion-ciclo
15 ;; NATURALEZA: Pura (misma entrada, misma salida)
16 ;; ESTRATEGIA: Condicional (evalua distintos casos para una entrada)
17 ;; IMPACTO: No destructiva
18 ;; =====
19 (defun recomendacion-ciclo (duracion)
20   (cond ((not (numberp duracion)) (error "Tipo de dato incorrecto. Debe ser numérico"))
21         ((not (plusp duracion)) (error "El numero debe ser mayor o igual a 0 (positivo)")) ;tambien puede ser (< ciclo-seg 0)
22         (t
23          (cond ((> duracion 130) "Ciclo demasiado largo")
24                ((< duracion 35) "Ciclo demasiado corto")
25                (t "Ciclo optimo")
26          )); fin primer cond
27   );fin cond
28 );fin

```



```
PS E:\tp_Integrador2026_grupo17\codigoLisp_ver2> sbcl
This is SBCL 2.6.4, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>.

SBCL is free software, provided as is, with absolutely no warranty.
It is mostly in the public domain; some portions are provided under
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (load "requerimiento4.lisp")
T
* (duracion-ciclo)
216
* (recomendacion-ciclo (duracion-ciclo))
"Ciclo demasiado largo"
* []
```

VERSIÓN 3:

Teniendo en cuenta la Fase 2, se integró el ecosistema de paquetes de **Quicklisp** para incorporar la librería **cl-json**. Esta librería permite el parseo de archivos en formato JSON, facilitando la gestión de los datos para que no estén fijos en el código, sino que se carguen dinámicamente desde un archivo .json de configuración externa.

Se desarrolló la función auxiliar **cargar-configuraciones**, la cual transforma el archivo **config.json** (a partir de la función **decode-json-from-source**) en una **lista de asociación (alist - lista de parejas)** de Lisp, donde cada clave del JSON se mapea a un **keyword** (símbolo de acceso global, ej: **:rojo**) vinculado a su respectivo valor numérico.

Para los datos, la función **duracion-ciclo** implementa una composición de funciones de acceso a listas de asociación: **"assoc"** que localiza el par clave-valor dentro de la lista **config** utilizando el **keyword** correspondiente (ej: **:rojo**). Y **"cdr"** que extrae el valor numérico asociado a dicha clave.

```
{ } config.json > ...
1  {
2    "rojo": 90,
3    "amarillo": 6,
4    "verde": 120
5  }
```

Bug 5: ADVERTENCIA VARIABLES NO DEFINIDAS

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

* (load "requerimiento4_3.lisp")

; file: E:\tp_Integrador2026_grupo17\codigoLisp_ver2\requerimiento4_3.lisp
; in: DEFUN DURACION-CICLO
;   (ASSOC AMARILLO CONFIG)
;
; caught WARNING:
;   undefined variable: COMMON-LISP-USER::AMARILLO
;
; caught WARNING:
;   undefined variable: COMMON-LISP-USER::CONFIG
;
;   (ASSOC VERDE CONFIG)
;
; caught WARNING:
;   undefined variable: COMMON-LISP-USER::CONFIG
;
;   (ASSOC ROJO CONFIG)
;
```

Error: las variables "rojo, verde o amarillo" no estan declaradas

Motivo: Al escribir rojo, verde o amarillo a secas dentro del `assoc`, Lisp interpreta que se hace referencia a una **variable** con alguno de esos nombres. Como no existe ninguna variable con esos nombres, el compilador te tira los *WARNINGS* y el debugger explota diciendo que las variables no están ligadas (*unbound*).

Solución: usar **Keywords** (los que llevan dos puntos adelante: `:rojo`, `:verde`, `:amarillo`), para buscar en el JSON parseado por `cl-json`.

Bug 6: undefined variable: COMMON-LISP-USER::CONFIG

```
;
; caught WARNING:
;   undefined variable: COMMON-LISP-USER::CONFIG

;   (ASSOC :VERDE CONFIG)
;
; caught WARNING:
;   undefined variable: COMMON-LISP-USER::CONFIG
; caught WARNING:
;   undefined variable: COMMON-LISP-USER::CONFIG
```

Error: no se reconoce la variable `config` (lista donde estan los valores de los colore)

Motivo: como la función `duracion-ciclo` esta declarada sin parámetros. Adentro intenta usar la variable `config`, pero como no está declarada de forma global o como argumento de la función Lisp no sabe de dónde sacarla.

Solución: `duracion-ciclo` tiene que recibir a `config` como parámetro obligatoriamente.

Bitácora de Depuración – Requerimiento 5 - Lenguaje: Common Lisp

Función Analizada: `cantidad-ciclos`

Objetivo: Desarrollar una función que determine cuántos ciclos completos de semáforo pueden ejecutarse en un período dado (minutos), aplicando validaciones robustas de entrada, conversión correcta de unidades temporales y acceso dinámico a las duraciones configuradas, garantizando resultados matemáticamente válidos dentro del dominio del problema.

Desarrollo: Esta función representa la consolidación práctica del paradigma funcional, transformando conceptos teóricos en una solución operativa que integra múltiples componentes del sistema. A diferencia del Requerimiento 4 (cálculo estático), aquí se materializa la interacción entre validación de datos, gestión de dependencias externas y cálculos temporales precisos.

El desarrollo reveló que la complejidad no reside en la fórmula matemática `(floor (/ (* minutos 60) duración-total))`, sino en los cuatro pilares de robustez que la sustentan:

1. **Contratos funcionales explícitos:** El primer error (`TYPE-ERROR`) expuso la necesidad de definir claramente qué retorna cada función auxiliar. Asumir que ``obtener-color`` devuelve un par cons sin verificarlo provocó fallos inmediatos al aplicar ``cdr`` sobre un entero. La solución estableció que las funciones auxiliares deben documentar su tipo de retorno antes de ser consumidas.

2. **Coherencia dimensional:** El segundo error (`LOGIC-ERROR`) demostró que las operaciones matemáticas son inservibles si las unidades son incompatibles. Multiplicar por 60 para convertir a segundos sólo tiene sentido si el denominador también está en segundos. Este bug silencioso (sin excepción, solo resultados erróneos) fue el más peligroso, evidenciando que la ausencia de errores de compilación no garantiza corrección lógica.

3. **Portabilidad de dependencias:** El tercer error (`FILE-ERROR`) surge al usar rutas relativas para cargar ``funcionesAux.lisp``. La solución con ``load-true name`` garantiza que el archivo se cargue desde el mismo

directorio del código fuente, independientemente del directorio de trabajo del REPL, resolviendo problemas de despliegue en diferentes entornos.

4. Validación semántica completa: El cuarto error (VALIDATION-ERROR) reveló que verificar el tipo (entero) es insuficiente; debe verificarse también el dominio (positivo > 0). Valores como 0 o -5 son enteros válidos sintácticamente, pero carecen de sentido en planificación temporal.

Código Fuente Analizado

```
λ cantidad-ciclos.lisp x
1  ;; =====
2  ;; REQUERIMIENTO 5: PLANIFICACIÓN TEMPORAL
3  ;; FUNCIÓN: cantidad-ciclos
4  ;; Entrada: minutos
5  ;; Salida: número de ciclos completos en ese período
6  ;; =====
7  (load "funcionesAux.lisp")
8
9  (defun cantidad-ciclos (minutos)
10    (if (not (integerp minutos))
11        "ingresar un numero entero"
12        (floor (* minutos 60)
13                (+ (cdr (obtener-color :rojo))
14                  (cdr (obtener-color :verde))
15                  (cdr (obtener-color :amarillo))))))
16  )
17  )
```

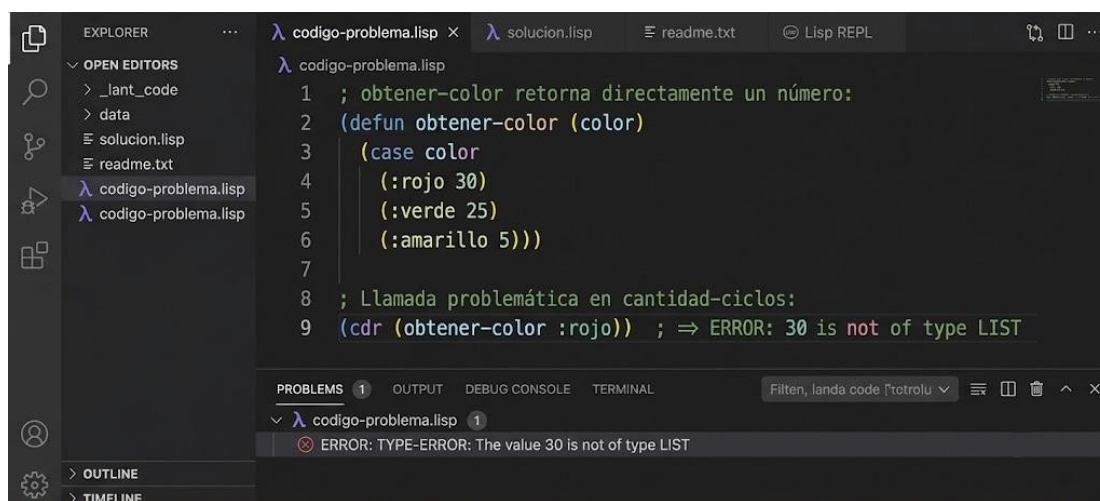
Errores documentados

Se identificaron y resolvieron 4 bugs conceptuales durante el desarrollo de la función **cantidad – ciclos**.

1 TYPE-ERROR

Uso de `cdr` sobre un valor no-cons – Error de tipo en tiempo de ejecución.

DESCRIPCIÓN DEL ERROR: Dentro de `cantidad-ciclos`, se aplica `(cdr (obtener-color :rojo))` asumiendo que `obtener-color` retorna un par cons del tipo `(color . duración)`. Sin embargo, si `obtener-color` está implementada para retornar directamente un valor numérico entero (la duración en segundos), aplicar `cdr` sobre un entero causa inmediatamente un error de tipo en tiempo de ejecución. Common Lisp exige que el argumento de `cdr` sea una lista o par cons; un número entero no satisface esa condición.



CAUSA: Se asumió erróneamente que `obtener-color` retornaba una estructura cons `(color . duración)`, como en `(:rojo . 30)`. Esta confusión surge de no haber definido explícitamente el contrato de retorno de la función auxiliar antes de usarla. Al llamar `(cdr 30)`, el intérprete lanza un `TYPE-ERROR` porque `cdr` sólo opera sobre cons cells o `nil`.

SOLUCIÓN APLICADA: Se rediseñó `obtener-color` para que retorne explícitamente un cons cell usando `(cons color duración)`, de forma que `cdr` pueda extraer la duración correctamente. Alternativamente, si `obtener-color` debe retornar solo el número, se elimina el `cdr` y se llama la función directamente.

```
λ código-corregido.lisp ×
λ código-corregido.lisp
1 ; OPCIÓN A: obtener-color retorna un cons cell
2 (defun obtener-color (color)
3   (case color
4     (:rojo (cons :rojo 30))
5     (:verde (cons :verde 25))
6     (:amarillo (cons :amarillo 5))))
7
8 ; Ahora cdr extrae correctamente la duración:
9 (cdr (obtener-color :rojo)) ; => 30 ✓
10
11 ; OPCIÓN B: acceso directo sin cdr
12 (defun obtener-color (color)
13   (case color
14     (:rojo 30)
15     (:verde 25)
16     (:amarillo 5)))
17
18 ; cantidad-ciclos sin cdr:
19 (+ (obtener-color :rojo) (obtener-color :verde) (obtener-color :amarillo))]
```

2 LOGIC-ERROR

Inconsistencia de unidades – Multiplicación `(* minutos 60)` incorrecta

DESCRIPCIÓN DEL ERROR: La expresión `(* minutos 60)` convierte minutos a segundos antes de dividir por la suma de duraciones de los colores. Sin embargo, si `obtener-color` retorna sus duraciones también en minutos (no en segundos), la conversión es incorrecta y produce resultados inflados por un factor de 60. Por ejemplo, con 2 minutos de entrada y duraciones de 1 minuto por color (total 3 min), el cálculo haría `(floor 120 3) => 40 ciclos` en lugar del correcto `(floor 2 3) => 0 ciclos`. El bug no lanza excepción, simplemente retorna un valor numéricamente absurdo.

```
λ código-problema.lisp ×
λ código-problema.lisp
1 ; Si obtener-color retorna duraciones en MINUTOS:
2 ; :rojo = 1, :verde = 1, :amarillo => 1 (total = 3 minutos)
3
4 (defun cantidad-ciclos (minutos)
5   (floor (* minutos 60) ; convierte a segundos
6         (+ (cdr (obtener-color :rojo)) ; suma en minutos
7           (cdr (obtener-color :verde))
8           (cdr (obtener-color :amarillo)))))
9
10 ; Con minutos = 2:
11 ; Numerador: 2 * 60 = 120 (segundos)
12 ; Denominador: 1 + 1 + 1 = 3 (minutos, no segundos)
13 ; Resultado: (floor 120 3) => 40 ; ¡incorrecto!
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL Filtrar, lanza code [tetrolu]

λ código-problema.lisp 1

ERROR: (cantidad-ciclos 2) => 40 ; debería ser 0 (menos de un ciclo completo)

CAUSA: No se estableció una unidad de medida uniforme entre la entrada `minutos` y los valores retornados por `obtener-color`. La función `cantidad-ciclos` aplica `(* minutos 60)` para convertir a segundos, pero si el denominador está en minutos, las dos magnitudes son incompatibles. Este es un error lógico clásico de conversión de unidades que no genera excepción, pero corrompe el resultado.

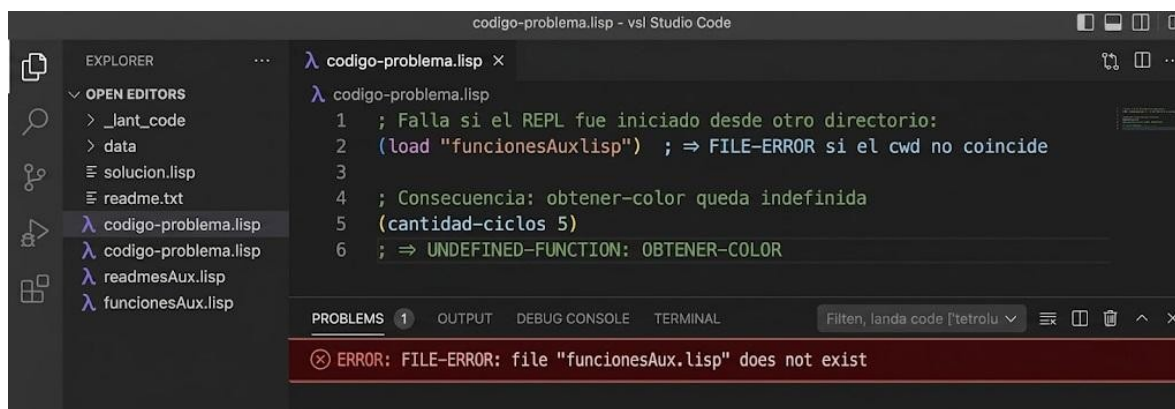
SOLUCIÓN APLICADA: Se unificaron las unidades. Como las duraciones de los semáforos suelen expresarse en segundos, se mantuvo la conversión `(* minutos 60)` y se corrigió `obtener-color` para que sus valores también estén en segundos. Alternativamente, si se trabaja en minutos, se elimina la multiplicación por 60.


```
λ codigo-corregido.lisp x
λ codigo-corregido.lisp
1 ; obtener-color retorna duraciones en SEGUNDOS (unidad unificada)
2 (defun obtener-color (color)
3   (case color
4     (:rojo (cons :rojo 30)) ; 30 segundos
5     (:verde (cons :verde 25)) ; 25 segundos
6     (:amarillo (cons :amarillo 5)))) ; 5 segundos
7
8 ; cantidad-ciclos: minutos * 60 ⇒ segundos / ciclo-total-segundos
9 (defun cantidad-ciclos (minutos)
10  (if (not (integerp minutos))
11      "ingresar un numero entero"
12      (floor (* minutos 60) ; segundos totales
13              (+ (cdr (obtener-color :rojo)) ; 30
14                  (cdr (obtener-color :verde)) ; 25
15                  (cdr (obtener-color :amarillo)))))) ; 5 ⇒ total 60s
16
17 ; Con minutos = 2: (floor 120 60) ⇒ 2 ciclos ✓
```

3 FILE-ERROR

Fallo en `(load "funcionesAux.lisp")` por ruta relativa ambigua

DESCRIPCIÓN DEL ERROR: La directiva `(load "funcionesAux.lisp")` en el nivel superior del archivo intenta cargar el archivo auxiliar usando una ruta relativa. Esta ruta se resuelve contra el directorio de trabajo actual del proceso Lisp (el REPL o el compilador), que varía según desde dónde se lanza el intérprete. Si el archivo se carga desde un directorio diferente al que contiene `funcionesAux.lisp`, la carga falló con un error de archivo no encontrado, dejando `obtener-color` sin definir. Cualquier llamada posterior a `cantidad-ciclos` provocará un error de símbolo no definido.



The screenshot shows the VS Code interface with a file named 'codigo-problema.lisp' open. The code in the editor is as follows:

```
λ codigo-problema.lisp x
λ codigo-problema.lisp
1 ; Falla si el REPL fue iniciado desde otro directorio:
2 (load "funcionesAux.lisp") ; ⇒ FILE-ERROR si el cwd no coincide
3
4 ; Consecuencia: obtener-color queda indefinida
5 (cantidad-ciclos 5)
6 ; ⇒ UNDEFINED-FUNCTION: OBTENER-COLOR
```

The 'PROBLEMS' panel at the bottom shows the following error:

```
ERROR: FILE-ERROR: file "funcionesAux.lisp" does not exist
```

CAUSA: En Common Lisp se resuelve rutas relativas en `load` contra `\*default-pathname-defaults\*`, que equivale al directorio de trabajo del proceso. Al cambiar el directorio desde el cual se lanza el REPL o al compilar desde un script de otro directorio, la ruta relativa deja de ser válida. El error se manifestó durante pruebas en diferentes terminales y sistemas operativos.

SOLUCIÓN APLICADA: Se reemplazó la ruta literal por una ruta construida dinámicamente usando `\*load-truename\*`, que contiene la ruta absoluta del archivo que está siendo evaluado. Esto garantiza que `funcionesAux.lisp` siempre se busque en el mismo directorio que el archivo que lo referencia, independientemente del directorio de trabajo.

```

λ código-corregido.lisp x
λ código-corregido.lisp
1 ; Solución: ruta relativa al archivo actual con *load-truename*
2 (load (merge-pathnames "funcionesAux.lisp"
3                       *load-truename*))
4
5 ; *load-truename* contiene la ruta absoluta del archivo actual.
6 ; merge-pathnames combina el directorio de ese archivo con el
7 ; nombre "funcionesAux.lisp", produciendo una ruta absoluta válida.
8
9 ; Alternativa: usar una ruta absoluta explícita (menos portable)
10 ; ([load "/ruta/absoluta/al/proyecto/funcionesAux.lisp"])

```

4 VALIDATION-ERROR

Validación incompleta - `integerp` acepta enteros negativos y cero

DESCRIPCIÓN DEL ERROR: La validación `(if (not (integerp minutos)) ...)` solo verifica que el argumento sea de tipo entero, pero no restringe el rango semántico válido del problema. Valores como `-5`, `-1` y `0` son enteros válidos según `integerp` y pasan la validación sin error. Sin embargo, un período de `0` minutos siempre producirá 0 ciclos (resultado trivial e inútil), y un período negativo producirá 0 o un número negativo de ciclos, lo cual carece completamente de sentido en el dominio de planificación temporal de semáforos.

```

λ código-problema.lisp x
λ código-problema.lisp
1 (defun cantidad-ciclos (minutos)
2   (if (not (integerp minutos)) ; solo valida tipo, no rango
3       "ingresar un numero entero"
4       (floor (* minutos 60) 60)))
5
6 ; Casos que pasan la validación pero son semánticamente inválidos:
7 (cantidad-ciclos 0) ; => 0 (trivial, sin sentido práctico)
8 (cantidad-ciclos -3) ; => -3 (negativo, imposible en el dominio)
9 (cantidad-ciclos -60) ; => -60 (absurdo)

```

ERROR: (cantidad-ciclos -3) => -3 ; resultado semánticamente inválido

CAUSA: Se confundió la validación de tipo con la validación de dominio. En el contexto del problema (planificación temporal de ciclos de semáforo), el argumento `minutos` debe ser un entero estrictamente positivo. Solo se aplicó la restricción de tipo (entero) omitiendo la restricción de dominio (positivo mayor a cero). El bug no genera excepción de Lisp, pero produce resultados incorrectos silenciosamente.

SOLUCIÓN APLICADA: Se amplió la condición de validación para incluir la verificación de que el entero sea estrictamente mayor a cero. Se usó `(or (not (integerp minutos)) (<= minutos 0))` como condición de error, cubriendo tanto el tipo incorrecto como el rango inválido en una sola expresión.

```

λ codigo-corregido.lisp x
1 (defun cantidad-ciclos (minutos)
2   (if (or (not (integerp minutos)) ; tipo incorrecto
3         (≤ minutos 0)) ; rango inválido (≤ 0)
4       "ingresar un numero entero positivo"
5       (floor (* minutos 60)
6               (+ (cdr (obtener-color :rojo))
7                 (cdr (obtener-color :verde))
8                 (cdr (obtener-color :amarillo))))))
9
10 ; Casos de prueba con la validación corregida:
11 (cantidad-ciclos 0) ; ⇒ "ingresar un numero entero positivo" ✓
12 (cantidad-ciclos -3) ; ⇒ "ingresar un numero entero positivo" ✓
13 (cantidad-ciclos 2) ; ⇒ 2 ✓ (resultado válido)

```

Bitácora de depuración : Requerimiento 6

Problema detectado

Durante las pruebas de la función `porcentaje-color`, se observó que los porcentajes calculados eran incorrectos. La salida obtenida era similar a:

```

❖ ((ROJO . 77.33333) (VERDE . 77.33333) (AMARILLO . 77.33333))

```

La suma de los porcentajes era aproximadamente 231%, cuando debería ser cercana al 100%.

Análisis

Se revisó la función `porcentaje-color` y se detectó que era invocada de la misma forma para los tres colores:

```

(defun porcentaje-color ()
  (let* ((rojo (cdr (obtener-color :rojo)))
         (verde (cdr (obtener-color :verde)))
         (amarillo (cdr (obtener-color :amarillo)))
         (ciclo (+ rojo verde amarillo))
         (resto (rem 3600 ciclo)))
    (list (cons 'rojo (calcular-rem resto ciclo rojo amarillo verde))
          (cons 'verde (calcular-rem resto ciclo rojo amarillo verde))
          (cons 'amarillo (calcular-rem resto ciclo rojo amarillo verde)) )
  )
)

```

Como la función `calcular-rem` recibía exactamente los mismos parámetros en cada llamada, no tenía forma de saber si debía calcular el porcentaje correspondiente al color rojo, verde o amarillo, por ende devolvía los mismos porcentajes para cada color.

Además, la función sólo consideraba el color donde terminaba el tiempo restante (`resto`) y no distribuía correctamente dicho tiempo entre los distintos colores del ciclo.

Solución aplicada

Se agregó un parámetro adicional denominado `actual` para indicar explícitamente el color a procesar:

```
(defun calcular-rest (resto rojo verde amarillo actual)
```

Como consecuencia, se modificó la invocación de la función `calcular-rem` añadiendo símbolos de cada color como parámetro, así estableciendo el valor de retorno correspondiente a cada color.

```
(cons 'rojo (calcular rojo (calcular-rest resto rojo verde amarillo 'rojo) ciclos))  
(cons 'verde (calcular verde (calcular-rest resto rojo verde amarillo 'verde) ciclos))  
(cons 'amarillo (calcular amarillo (calcular-rest resto rojo verde amarillo 'amarillo) ciclos))
```

También se modificó la lógica de `calcular-rem` para devolver únicamente los segundos adicionales correspondientes al color solicitado.

Resultado

Luego de aplicar las correcciones, la función produjo resultados coherentes:

```
((ROJO . 42.5) (VERDE . 54.833336) (AMARILLO . 2.666667))
```

La suma de los porcentajes obtenidos es aproximadamente 100%, cumpliendo con el comportamiento esperado para una distribución temporal sobre un período de una hora.

Conclusión

El error se originaba porque la función auxiliar no distinguía qué color estaba calculando. La incorporación del parámetro `actual` y la corrección del cálculo del tiempo restante permitieron obtener una distribución porcentual correcta para cada color del semáforo.